

Vacuity Auditor

Can this check ever go red?

A per-claim auditor for AI-assisted code review. It audits the check, not the code the check is pointed at, and answers in one of four words.

IBM Bob 2.0 Hackathon | lablab.ai | 2026-09-25 to 2026-09-27

Solo entry - Wang Jian - github.com/jianwang-ntu/ibm-bob-2-hackathon

STATUS, STATED FIRST

IBM Bob 2.0 has not been used on this project. Access opens at kickoff and nothing here claims otherwise.

Every figure here is generated from evidence/*.json and re-checked against these PDF bytes.

A green tick that cannot go red is read as evidence

- Code review of AI-assisted changes - one of the workflows the challenge brief names by hand, alongside onboarding, debugging and testing.
- When one agent writes a change AND the test that verifies it, the test very often cannot fail. assert result is not None passes for every implementation that returns an object.
- An assertion inside try: ... except: pass passes for all of them. A suite that collected zero tests exits green, and pytest exit 5 looks like success to a CI badge.
- A reviewer sees a green tick and approves. The tick meant nothing, and no tool on the shelf says so: they all answer did the suite pass?, not could it have failed?

MEASURED, NOT ASSERTED

In this repository's worked example two green suites cover the same four-line function: 8 / 9 against 2 / 9, and pytest calls both green.

Two green suites. One is evidence, one is decoration.

examples/demo_project - one 4-line pricing function, two test files, both passing under pytest

CLAIM	VERDICT	KILLED	WHAT THAT MEANS
real_suite_verifies_pricing	WEAK	8 / 9	evidence, with named gaps
vacuous_suite_verifies_pricing	WEAK	2 / 9	green for almost any wrong code

Neither is clean, and that is the honest answer. The real suite never tests a 0% discount, so a mutant moving its guard survives it - a gap in a test its author believed complete. The split, and the named survivors, are the signal; a single pass/fail letter is not.

Survivors the vacuous suite accepts, verbatim from the report:

```
swap_arithmetic@src/pricing.py:11
swap_arithmetic@src/pricing.py:12
swap_boollop@src/pricing.py:9
perturb_number@src/pricing.py:11
```

Four stages, and the order is load-bearing

1 Green-baseline guard

Before any mutation the unmutated tree must be green, and green is checked properly: pytest exits 5 on zero collected and 4 on a usage error. A campaign layered on either scores 100% for free.

2 Static vacuity scan

AST patterns that make a check unable to go red - no_assertion, presence_only, self_comparison, swallowed_assertion and three more. These are candidates, never verdicts. They locate; stage 3 decides.

3 Mutation as a negative control

Mutants are built by rewriting the AST and unparsing, so every one is valid Python. Docstrings and log strings are excluded: they change no behaviour and would brand correct checks VACUOUS.

4 Two guards on the result

A mutant that stops the check RUNNING looks like a kill from outside; those runs leave both numerator and denominator. And each sandbox refuses to be an existing directory, which is where nested-copy 100% come from.

Stages 1 and 4 are the whole point. Every false 100% this tool has met came from a campaign that scored a suite which never ran, or from a copy that nested itself one directory deeper and left the check reading a stale tree.

Four words, and three of them are not a pass

DISCRIMINATING

`exit 0`

every behaviour-changing mutation of the claimed region turned the check red

WEAK

`exit 3`

some did, and named survivors remain: the check accepts specific wrong implementations

VACUOUS

`exit 1`

nothing did. Green from this check is not evidence for this claim

INCONCLUSIVE

`exit 2`

the audit could not reach a conclusion. Never reported as a pass

Each band has its own exit code on purpose: not evidence, could not tell and evidence with gaps call for three different actions, and collapsing them into pass/fail is what makes a review tick meaningless in the first place.

It audits itself, and the result is not flattering

```
vacuity-audit audit --root . --spec vacuity.toml (--max-mutants 10)
```

CLAIM	VERDICT	KILLED / EVIDENTIAL
baseline_guard_reads_baseline	WEAK	3 / 8
static_scan_tests_read_static_scan	WEAK	7 / 10
runner_tests_read_the_isolation_code	WEAK	7 / 10
mutant_tests_read_the_operators	WEAK	9 / 10
report_tests_read_the_exit_codes	WEAK	8 / 10

5 of 5 claims come back WEAK, 0 DISCRIMINATING. The worst is baseline_guard_reads_baseline at 3 / 8. Its tests match reason strings by substring, so appending to one does not turn them red - a gap the tool found in its own suite.

In this repository's own tests it flags 5 candidates, of which 1 is a no_assertion true positive.

Nothing here is DISCRIMINATING. That is a real finding about this repository's own tests, the survivors are published in evidence/self_audit.json, and it is stated rather than hidden.

The number moves with the budget, and we say so

At a budget of 4 the same suite reports DISCRIMINATING 4 / 4; at 12 it reports WEAK 8 / 9.

Same suite, same module, same tool. --max-mutants caps the pool and the cap is taken in generation order, not by sampling.

- A small budget can turn a real gap into a clean bill of health. DISCRIMINATING at a low budget means no counterexample found yet, not none exists.
- This is on the slide, in the README and in the tool's own limitations list. It is the single most misleading thing the tool can output and it is not implemented away.
- Fixing it properly - warning when the cap binds, and sampling instead of truncating - is named on the roadmap and is not built.

Prior art is older; the question is different.

MUTATION TESTING TODAY

- mutmut, cosmic-ray and Pitest all do the mutation itself better than this does.
- They score a SUITE: one quality number over a whole test tree.
- None pairs a single check with the module it claims to verify.

WHAT THIS ADDS

- A PER-CLAIM question: does this check read this module?
- A green-baseline guard, so a suite that never ran cannot score.
- The did-not-run exclusion and the sandbox-nesting refusal - where false 100%s actually come from.

Stated against ourselves: on raw mutation quality the incumbents win, and a team that only wants a suite-level score should use them. The claim here is narrower - a per-claim verdict with named survivors is what an AI-assisted review needs, and the guards above are what keep the number honest.

Also unlike them, this tool publishes its own audit and comes back WEAK on every claim rather than shipping a green badge.

Who it is for, and what it changes

- The user is a reviewer or a tech lead merging AI-written changes, where the volume of generated tests is what makes reading them all impossible.
- The change is at the merge gate: a check labelled VACUOUS stops being read as evidence, and attention goes where the suite is provably blind.
- The cost avoided is a defect shipped behind a green tick. This entry puts no currency figure on that, because it has measured none.

DEMAND EVIDENCE, AND ITS SAMPLE SIZE

The static scan flags 5 vacuity candidates in the worked example before anything is executed.

5 of 5 claims come back WEAK, 0 DISCRIMINATING.

One repository and one worked example: far too small to generalise from.

No market study was conducted. The only demand evidence here is the base rate measured on this repository and its worked example.

The business model, labelled as a plan

WHO WOULD PAY, AND FOR WHAT

- Open-source core under MIT - the auditor itself, free.
- The paid surface would be a CI service: hosted runs, budget management, and drift alerts on a claim's verdict.
- The buyer is an engineering org with a merge gate.

WHAT WOULD VALIDATE IT

- Audit N public repositories and publish the base rate of VACUOUS claims. None of that is done.
- Then price against the review hours it redirects.
- Until both exist, any figure would be invented.

No revenue, no customers, no pricing validated. The business model here is a plan, not a result.

A deck that answers the market question with a large number it did not measure scores well and is not true. Nobody here has measured the review hours redirected either. This one answers with the model, the buyer and the two experiments that would test it, and leaves the number blank until they are run.

What comes next, and what Bob 2.0 is for

ROADMAP, IN PRIORITY ORDER

- Oracle sensitivity: swap a check's INPUT for noise and see whether the verdict moves at all.
- Warn when --max-mutants binds, and sample instead of truncating.
- Diff-scoping and an incremental mode, so a merge gate can run it.
- Languages beyond Python, which needs new AST operators per language.

THE 44-HOUR WINDOW, WITH BOB 2.0

- Bob 2.0 access opens at kickoff; nothing in this repository has used it and nothing here claims it has.
- The planned use is Agent mode and subagents on the roadmap items opposite, with every Bob-assisted file disclosed as the rules require.
- The auditor will then be pointed at Bob's own output, which is the sharpest test of the idea available.

Oracle sensitivity - swapping a check's input for noise - is designed and not implemented. It is in no number here.

Python only. The AST operators and the pytest exit-code semantics read no other language.

What is not built

- IBM Bob 2.0 has not been used on this project. Access opens at kickoff and nothing here claims otherwise.
- Nothing in this repository's own audit is DISCRIMINATING: 5 of 5 claims are WEAK.
- Oracle sensitivity - swapping a check's input for noise - is designed and not implemented. It is in no number here.
- Python only. The AST operators and the pytest exit-code semantics read no other language.
- The verdict depends on the mutant budget and the tool does not warn you. Treat DISCRIMINATING at a low budget as no counterexample found yet.
- No revenue, no customers, no pricing validated. The business model here is a plan, not a result.
- No market study was conducted. The only demand evidence here is the base rate measured on this repository and its worked example.
- No hosted demo URL and no video yet: both need accounts this entry does not hold.

An honest smaller tool beats an inflated description of a larger one, and a project whose subject is unfalsifiable claims does not get to make any.

Reproduce every number on these slides

```
git clone https://github.com/jianwang-ntu/ibm-bob-2-hackathon
cd ibm-bob-2-hackathon && pip install -e .
vacuity-audit audit --root . --spec vacuity.toml
vacuity-audit audit --root examples/demo_project --spec vacuity.toml
```

THE REPOSITORY, MEASURED FROM OUTSIDE

30 tracked files, 10 modules, 6 test files, 64 tests passing in a clean anonymous clone at 2259a4e8.

5 commits, 1 commit author, MIT licensed.

Counted in a fresh anonymous clone by `collect_repo_state.py`, not read off the README.

Wang Jian - solo entry - IBM Bob 2.0 Hackathon | lablab.ai | 2026-09-25 to 2026-09-27